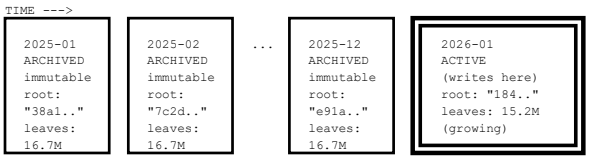
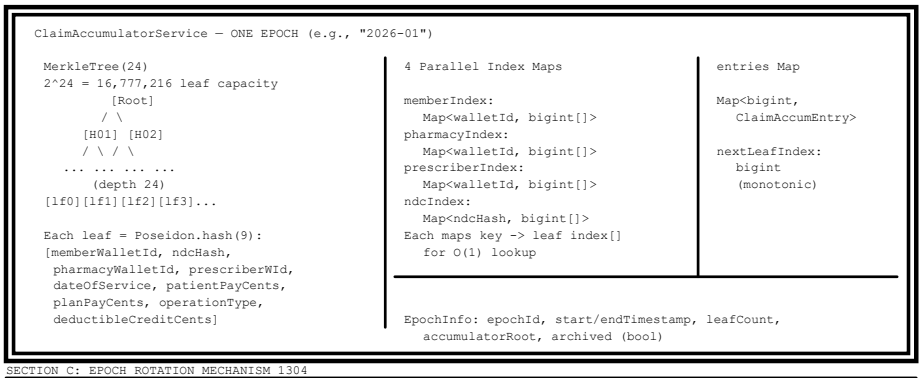
FIG. 13 — EPOCH-BASED TEMPORAL ACCUMULATOR 1300

SECTION A: EPOCH TIMELINE WITH MONTHLY ROTATION 1300

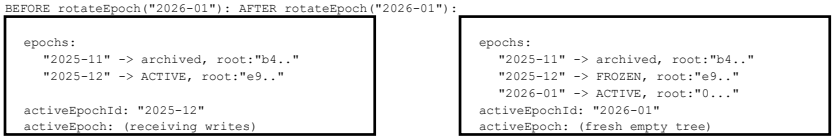


Each epoch = 1 ClaimAccumulatorService (MerkleTree(24)+4 indexes+entries). YYYY-MM format.
Production: tree height scales with volume (e.g., height 25 = 33.5M, or sub-monthly rotation).

SECTION B: SINGLE EPOCH INTERNAL STRUCTURE 1302



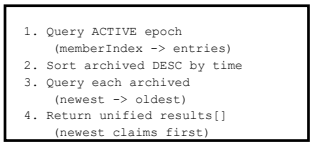
SECTION C: EPOCH ROTATION MECHANISM 1304



Step 1: FREEZE "2025-12" — set endTimestamp, archived=true, final leafCount+root
Step 2: ARCHIVE — archivedAccumulators.set("2025-12", activeEpoch)
Step 3: CREATE — new ClaimAccumulatorService() (empty tree, empty indexes, leafIndex=0)
Step 4: REGISTER — activeEpochId = "2026-01", epochs.set("2026-01", {archived:false,...})
Returns: EpochInfo of closed "2025-12" epoch

SECTION D: CROSS-EPOCH QUERY AGGREGATION 1306

getClaimsByMember(walletId) --> queryCrossEpoch(acc => acc.getClaimsByMember(wId))

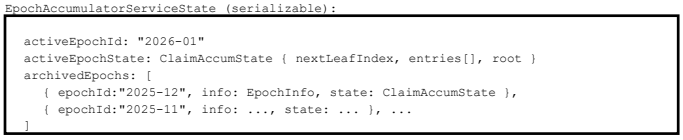


CROSS-EPOCH QUERIES:
getClaimsByMember(walletId)
getClaimsByPharmacy(walletId)
getClaimsByPrescriber(walletId)
getClaimsByNdc(ndcHash)
SINGLE-EPOCH:
getClaimsByMemberInEpoch(
walletId, epochId)
METADATA: -> epochId MUST be specified
getActiveRoot(), getRootForEpoch(),
getTotalLeafCount(), listEpochs()

EPOCH-SCOPED MERKLE PROOF:
getClaimWithProof(epochId,
leafIdx)
-> entry + witnessJson + root
-> proof verifies against
that epoch's FROZEN root
-> permanently valid
-> epochId MUST be specified
(no cross-epoch proofs)

SECTION E: CHECKPOINT PERSISTENCE 1308

EpochAccumulatorServiceState (serializable):



restoreState: 1.CLEAR all -> 2.RESTORE archived (rebuild trees+indexes) ->
3.RESTORE active -> 4.VERIFY roots match

SECTION F: PRODUCTION SCALE 1310

US scale: 250M claims/yr, ~20.8M/mo. MerkleTree(24) = 16.7M leaves; production uses larger trees
(height 25 = 33.5M capacity) or sub-monthly rotation to accommodate peak volumes.
25-year archive: 300 epochs, ~6.25B claims, ~4TB disk. Each epoch independently queryable+provable.
Active epoch root committed per L2Batch. Archived roots are fixed historical values.